If those aren't flexible enough, the slice_when method takes a block with two arguments, applies it to overlapping pairs of the list in turn, and splits the slice between the two elements if the block returns true. For example:

```
list = [1, 1, 2, 3, 5, 8, 13, 21]
list.slice_after { |i| i.even? }.to_a  # => [[1, 1, 2], [3, 5, 8], [13, 21]]
list.slice_before { |i| i.even? }.to_a # => [[1, 1], [2, 3, 5], [8, 13, 21]]

list = [18, 17, 3, 11, 3, 10]
list.slice_when { |a, b| a > b }.to_a  # => [[18], [17], [3, 11], [3, 10]]
```

Querying

The most basic query about a collection is "how big is it?" The Ruby method here is count, which without a block returns the size of the collection. With a block argument, it returns the number of elements in the collection for which the block returns a truthy value. With a positional argument, it returns the count of elements that are equal to that argument.

The tally method is a generalization of count. It returns a Hash, the keys of which are values in the list, and the values of which are the count of how many times that value appears in the list:

```
sample = [1, 2, 3, 2, 3, 1, 3, 5]
sample.count                # => 8
sample.count(2)             # => 2
sample.count { |x| x.even? } # => 2
sample.tally                # => {1=>2, 2=>2, 3=>3, 5=>1}
```

The include? method (aliased as member?) takes an argument and returns true if there is an element in the collection that is equal to that argument.

A series of methods returns true or false based on how many times a block or pattern is matched by the collection. All of these methods behave the same way: they take a block or a positional argument. With a positional argument, the method counts how many elements of the list are === to that argument. With a block, the method counts how many elements of the list cause the block to return true. Given that count:

- none? returns true if and only if the count is zero.
- one? returns true if and only if the count is one.
- any? returns true if the count is one or more.
- all? returns true if the count is the size of the list (all elements of the list match).

Sorting and Comparing

When you create a new object, you often want to be able to sort a list of those objects or compare two objects along some default criteria.

The most general feature in Ruby for sorting is the Enumerable#sort method, the receiver of sort is the item to be sorted, which can be any object that includes the Enumerable module. For our purposes, the most important class that can respond to sort is Array.

If sort is called without a block, it calls <=> on each element, which is to say the following two lines are identical:

```
list_of_users.sort { |a, b| a <=> b }
list_of_users.sort
```